

A DYNAMIC PROGRAMMING APPROACH TO SEQUENCING PROBLEMS

Michael Held, IBM Data Systems Division, New York, N. Y.

Richard M. Karp, IBM Research, Yorktown Heights, N. Y.

Introduction

This paper explores a dynamic programming approach to the solution of three sequencing problems: a scheduling problem involving arbitrary cost functions, the traveling-salesman problem, and an assembly line balancing problem. Each of the problems is shown to admit of numerical solution through the use of a simple recursion scheme; these recursion schemes also exhibit similarities and contrasts in the structures of the three problems. For large problems, direct solution by means of dynamic programming is not practical, but procedures are given for obtaining good approximate results by solving sequences of smaller derived problems. Experience with a computer program for the solution of traveling-salesman problems is presented.

A Scheduling Problem

We consider a set of jobs $J_1, J_2, \dots, J_n$ which are to be executed on a single facility and have execution times $\tau_1, \tau_2, \dots, \tau_n$. With each job J_k is associated a function $c_k(t)$, giving the cost associated with completing the job at time t . Any given order of execution of the jobs determines a schedule, and fixes the time t_k at which each job J_k is completed. The cost associated with a schedule is $\sum_{k=1}^n c_k(t_k)$; we seek an order of execution which minimizes this cost.

Given $S \subseteq \{1, 2, \dots, n\}$ and $\ell \in S$, let $S - \ell$ denote the set obtained by deleting ℓ from S . Let $C(S)$ be the minimum cost incurred in executing the jobs with indices in S in any order whatever in the interval $[0, t_S]$ where

$t_S = \sum_{j \in S} \tau_j$. Then we may define the following recurrence relations:

$$(1) \quad \begin{cases} C(\{\ell\}) = c_\ell(\tau_\ell), \text{ for any } \ell \\ C(S) = \min_{\ell \in S} [C(S - \ell) + c_\ell(t_S)] . \end{cases}$$

It follows that an ordering $(i_1, i_2, \dots, i_n)$ is optimum if and only if, for $1 \leq p \leq n$,

$$(2) \quad C(\{i_1, i_2, \dots, i_p\}) = C(\{i_1, i_2, \dots, i_{p-1}\}) + c_{i_p}(\{i_1, i_2, \dots, i_p\}) .$$

The procedure for finding an optimum solution has two phases. In the first phase, the quantities $C(S)$, for all $S \subseteq \{1, 2, \dots, n\}$, are computed recursively from (1). In the second phase, (2) is used to generate an optimum ordering.

The Traveling-Salesman Problem

The traveling-salesman problem is that of finding a cyclic permutation $P = (1, i_2, i_1, \dots, i_n)$ of the integers from 1 through n (corresponding to cities to be visited) that minimizes the cost of a complete tour, given by

$$a_{1i_2} + a_{i_2i_3} + a_{i_3i_4} + \dots + a_{i_n1} ,$$

where the $a_{\alpha\beta}$ are a given set of real numbers (distances).

Given $S \subseteq \{2, 3, \dots, n\}$ and $\ell \in S$, let $C(S, \ell)$ denote the minimum cost of starting from city one and visiting all cities in the set S , terminating at city ℓ . Then

$$C(\{\ell\}) = a_{1\ell}, \text{ for any } \ell$$

$$C(S, \ell) = \min_{m \in S - \ell} [C(S - \ell, m) + a_{m\ell}] .$$

Finally, if C denotes the minimum cost of a complete tour,

$$C = \min_{\ell \in \{2, 3, \dots, n\}} [C(\{2, 3, \dots, n\}, \ell) + a_{\ell 1}] .$$

A permutation $(1, i_2, i_3, \dots, i_n)$ is optimum if, and only if,

$$C = C(\{2, 3, \dots, n\}, i_n) + a_{i_n 1} \text{ and, for } 2 \leq p \leq n - 1,$$

$$C(\{i_2, i_3, \dots, i_p, i_{p+1}\}, i_{p+1}) = C(\{i_2, i_3, \dots, i_p\}, i_p) + a_{i_p i_{p+1}} .$$

As before, a two-phase computation is used to obtain an optimum solution.*

An Assembly Line Balancing Problem

An assembly line is required to carry out, for each unit produced, a set of jobs $J_1, J_2, \dots, J_n$. A given job J_k requires τ_k units of time for its execution, and may be assigned to any of the locations (work stations) placed serially along the assembly line. The assignment is to be made so that:

- (1) t_i , the sum of the execution times of jobs assigned to the i th work station, does not exceed the cycle time T ;
- (2) a given set of precedence requirements are not violated;
- (3) the number of work stations is minimized.

The set $S \subseteq \{1, 2, \dots, n\}$ is said to be feasible if there is no pair (J_ℓ, J_m) such that (a) $\ell \notin S$, (b) $m \in S$, and (c) J_ℓ is required to precede J_m . With any feasible set S is associated a cost $C(S)$, which is the minimum, over all feasible assignments of the jobs in S , of the quantity $(r-1)T + t_r$, where r is the number of work stations needed in the assembly line according to a particular assignment. Then

$$C(\{\ell\}) = \tau_\ell$$

$$C(S) = \min_{\ell \in S \text{ and } S-\ell \text{ feasible}} [C(S-\ell) + \phi(C(S-\ell), \tau_\ell)]$$

where if $\left\lfloor \frac{x}{T} \right\rfloor = \left\lfloor \frac{x+y}{T} \right\rfloor$, $\phi(x, y) = y$

if $\left\lfloor \frac{x}{T} \right\rfloor \neq \left\lfloor \frac{x+y}{T} \right\rfloor$, $\phi(x, y) = T \left\lfloor \frac{x+y}{T} \right\rfloor + y - x$.

The derivation of an optimum solution from these recursion relation proceeds as in the previous algorithms.

*R. Bellman has given a nearly identical algorithm in his article, "Combinatorial Processes and Dynamic Programming," Proceedings of the Tenth Symposium in Applied Mathematics of the American Mathematical Society, April 24-26, 1958, published by the American Mathematical Society, 1960.

Successive Approximations

The algorithms which have been presented permit the rapid direct solution of problems of moderate size. Nevertheless, the complexity of these algorithms, measured by storage requirements or numbers of arithmetic operations, grows quite rapidly with the size of the problem. Problems beyond the scope of direct solution can be treated systematically by means of a successive approximations procedure. This procedure yields a sequence of permutations, each obtained from its predecessor by the solution of a derived subproblem of moderate size having the same structure as the given problem. The associated costs form a monotone nonincreasing sequence which may not converge to the optimum solution; however, computer experimentation has yielded excellent results in a variety of cases. The application of this approach to the traveling-salesman problem is discussed in detail.